DEDICATED TO INNOVATION

www.bacancy.com

BACANCY®

Customer Satisfaction Is Our Highest Priority

Employee Matters

In this session

What we are learning in this session

- Introduction of EF
- EF Features
- EF Architecture
- EF Installation
- Data Annotation

Let’s Start

Entity Framework - Introduction

- Prior to .NET 3.5, we (developers) often used to write ADO.NET code or Enterprise Data Access Block to save or retrieve application data from the underlying database. We used to open a connection to the database, create a DataSet to fetch or submit the data to the database, convert data from the DataSet to .NET objects or vice-versa to apply business rules. This was a cumbersome and error prone process. Microsoft has provided a framework called "Entity Framework" to automate all these database related activities for your application.
- Entity Framework is an open-source ORM framework for .NET applications supported by Microsoft. It enables developers to work with data using objects of domain specific classes without focusing on the underlying database tables and columns where this data is stored.
- With the Entity Framework, developers can work at a higher level of abstraction when they deal with data, and can create and maintain data-oriented applications with less code compared with traditional applications.

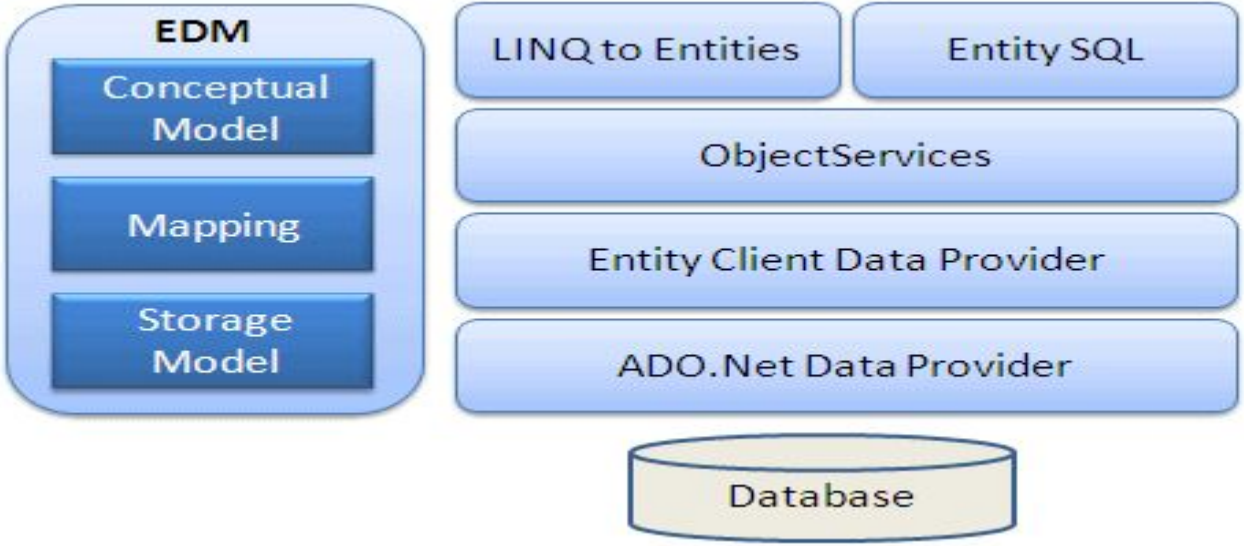
EF - Features

- **Cross-platform:** EF Core is a cross-platform framework which can run on Windows, Linux and Mac.
- **Modelling:** EF (Entity Framework) creates an EDM (Entity Data Model) based on POCO (Plain Old CLR Object) entities with get/set properties of different data types. It uses this model when querying or saving entity data to the underlying database.
- **Querying:** EF allows us to use LINQ queries (C#/VB.NET) to retrieve data from the underlying database. The database provider will translate this LINQ queries to the database-specific query language (e.g. SQL for a relational database). EF also allows us to execute raw SQL queries directly to the database.
- **Change Tracking:** EF keeps track of changes occurred to instances of your entities (Property values) which need to be submitted to the database.
- **Saving:** EF executes INSERT, UPDATE, and DELETE commands to the database based on the changes occurred to your entities when you call the SaveChanges() method. EF also provides the asynchronous SaveChangesAsync() method.
- **Concurrency:** EF uses Optimistic Concurrency by default to protect overwriting changes made by another user since data was fetched from the database

EF - Features

- **Transactions:** EF performs automatic transaction management while querying or saving data. It also provides options to customize transaction management.
- **Caching**: EF includes first level of caching out of the box. So, repeated querying will return data from the cache instead of hitting the database.
- **Built-in Conventions**: EF follows conventions over the configuration programming pattern, and includes a set of default rules which automatically configure the EF model.
- **Configurations:** EF allows us to configure the EF model by using data annotation attributes or Fluent API to override default conventions.
- **Migrations:** EF provides a set of migration commands that can be executed on the NuGet Package Manager Console or the Command Line Interface to create or manage underlying database Schema.

EF - Architecture



- **EDM (Entity Data Model):** EDM consists of three main parts - Conceptual model, Mapping and Storage model.

EF -Architecture

- **Conceptual Model:** The conceptual model contains the model classes and their relationships. This will be independent from your database table design.
- **Storage Model:** The storage model is the database design model which includes tables, views, stored procedures, and their relationships and keys.
- **Mapping**: Mapping consists of information about how the conceptual model is mapped to the storage model.
- **LINQ to Entities:** LINQ-to-Entities (L2E) is a query language used to write queries against the object model. It returns entities, which are defined in the conceptual model. You can use your LINQ skills here.
- **Entity SQL:** Entity SQL is another query language (For EF 6 only) just like LINQ to Entities. However, it is a little more difficult than L2E and the developer will have to learn it separately.
- **Object Service:** Object service is a main entry point for accessing data from the database and returning it back. Object service is responsible for materialization, which is the process of converting data returned from an entity client data provider (next layer) to an entity object structure.

EF -Architecture

- **Entity Client Data Provider:** The main responsibility of this layer is to convert LINQ-to-Entities or Entity SQL queries into a SQL query which is understood by the underlying database. It communicates with the ADO.Net data provider which in turn sends or retrieves data from the database.
- **ADO.Net Data Provider:** This layer communicates with the database using standard ADO.Net.

Diff. EF Core and EF 6

EF Core	EF 6
It is cross platform.	It is for windows only
Designed for .NET Framework.	Designed for .NET Core.
Optimized for performance.	Less Optimized.
Regular updates with new features	Limited updates.
Supports command batching.	Does not support command batching.

EF Core Installation

Package required :

- **Microsoft.EntityFrameworkCore.Tools**

  The Entity Framework Core tools help with design-time development tasks. They're primarily used to manage Migrations and to scaffold a DbContext and entity types by reverse engineering the schema of a database.

- **Microsoft.EntityFrameworkCore.SqlServer**

  Microsoft.EntityFrameworkCore.SqlServer is the EF Core database provider package for Microsoft SQL Server and Azure SQL.

EF Core Installation

- **DbContext :** The DbContext class is a core component of Entity Framework Core (EF Core) that acts as a bridge between your application's domain (entities) classes and the underlying database. It manages database connections, performs CRUD (Create, Read, Update, Delete) operations, manages transactions, and tracks changes to entities. The DbContext class is responsible for interacting with the database using the configured database provider (e.g., SQL Server, SQLite, etc.).
- **DbSet:** The DbSet<TEntity> class represents an entity set that can be used for create, read, update, and delete operations.

EF Core Installation

```
// Import the Entity Framework Core namespace to access DbContext and other EF Core functionalities.
using Microsoft.EntityFrameworkCore;

namespace EFCoreCodeFirstDemo.Entities
{
    // EFCoreDbContext class inherits from DbContext, which is the primary class for interacting with the database using EF Core.
    public class EFCoreDbContext : DbContext
    {
        // Constructor that accepts DbContextOptions<EFCoreDbContext> as a parameter.
        // The options parameter contains the settings required by EF Core to configure the DbContext,
        // such as the connection string and provider.
        public EFCoreDbContext(DbContextOptions<EFCoreDbContext> options)
        : base(options) // The base(options) call passes the options to the base DbContext class constructor.
        {
        }

        // DbSet<Student> Students represents a table in the database corresponding to the Student entity.
        // EF Core uses DbSet<TEntity> to track changes and execute queries related to the Student entity.
        public DbSet<Student> Students { get; set; }

        // DbSet<Branch> Branches represents a table in the database corresponding to the Branch entity.
        // Similar to the Students DbSet, this property is used by EF Core to track and manage Branch entities.
        public DbSet<Branch> Branches { get; set; }

        public DbSet<Product> Products { get; set;}
    }
}
```

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
{
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"));
});
```

```
"ConnectionStrings": {
  "DefaultConnection":
"Server=.\\SQLEXPRESS;Database=ProductCrud;Trusted_Connection=True;TrustServerCertificate=True;"
}
```

EF Core Migration

- **Create Migration:**

  CMD or Bash :- **dotnet ef migrations add migration-name**

  Package Manager Console :- **Add-Migration migration-name**

- **Update Database:**

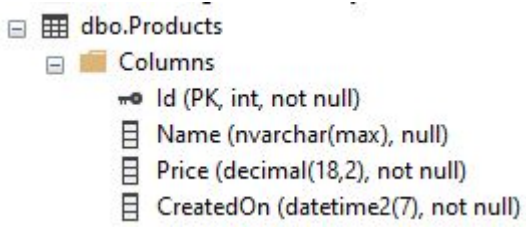
  CMD or Bash :- **dotnet ef database update**

  Package Manager Console :- **Update-Database**

EF - Model DataAnnotations

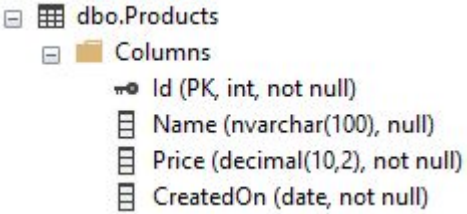
- Without DataAnnotations

```
public class Product
{
    0 references
    public int Id { get; set; }
    0 references
    public string Name { get; set; }
    0 references
    public decimal Price { get; set; }
    0 references
    public DateTime CreatedOn { get; set; }
}
```



- With DataAnnotations

```
public class Product
{
    [Key]
    0 references
    public int Id { get; set; }
    [MaxLength(100)]
    0 references
    public string Name { get; set; }
    [Column(TypeName = "decimal(10, 2)")]
    0 references
    public decimal Price { get; set; }
    [Column(TypeName = "date")]
    0 references
    public DateTime CreatedOn { get; set; }
}
```



Thank you